

RELAZIONE DI LABORATORIO

Controllo Accessi Discrezionale (DAC) su Kali Linux — S10/L2

Autore: Salvatore Friscaro

1. Obiettivo e contesto

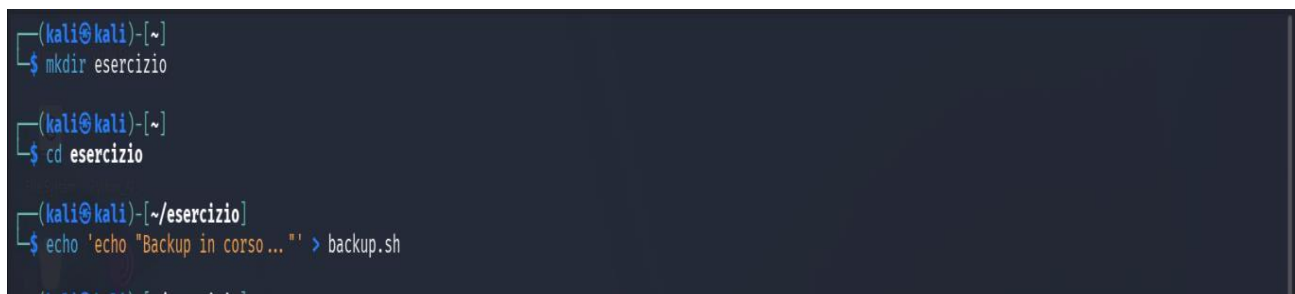
Questo laboratorio serve a mettere in pratica i permessi DAC (Discretionary Access Control) su un sistema Kali Linux. L'idea era di prendere un caso concreto — uno script di manutenzione chiamato `backup.sh`, dentro una cartella chiamata `esercizio` — e capire come restringerne l'accesso applicando il principio del privilegio minimo, cioè dare a ogni utente solo i permessi strettamente necessari.

2. Svolgimento e comandi eseguiti

Fase 1 — Creazione della cartella e dello script

```
mkdir esercizio && cd esercizio && echo 'echo "Backup in corso..."' > backup.sh
```

Con questo comando ho creato la cartella di lavoro `esercizio` e, di seguito, il file `backup.sh` contenente una semplice istruzione di `echo`, giusto per avere qualcosa da eseguire più avanti.



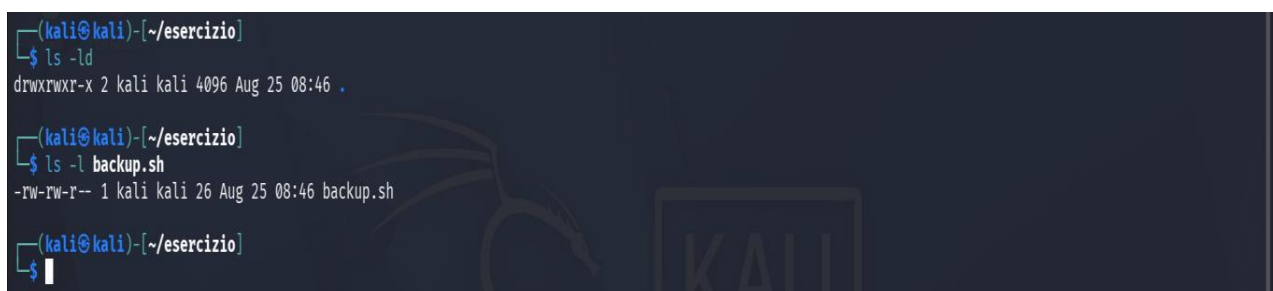
```
(kali@kali)~  
$ mkdir esercizio  
  
(kali@kali)~  
$ cd esercizio  
  
(kali@kali)~/esercizio  
$ echo 'echo "Backup in corso..."' > backup.sh
```

Fig. 1 — Creazione della cartella `esercizio` e del file `backup.sh`

Fase 2 — Controllo dei permessi di partenza

```
ls -ld .  
ls -l backup.sh
```

Prima di modificare qualsiasi cosa ho controllato quali permessi venivano assegnati di default, cioè quelli derivati dalla `umask` del sistema (0022). La cartella risulta `drwxrwxr-x` (775), mentre il file è `-rw-rw-r--` (664). Come si vede, in questo stato lo script non ha ancora il bit di esecuzione, quindi non potrebbe essere lanciato direttamente.



```
(kali@kali)~/esercizio  
$ ls -ld  
drwxrwxr-x 2 kali kali 4096 Aug 25 08:46 .  
  
(kali@kali)~/esercizio  
$ ls -l backup.sh  
-rw-rw-r-- 1 kali kali 26 Aug 25 08:46 backup.sh  
  
(kali@kali)~/esercizio  
$
```

Fig. 2 — Permessi iniziali di cartella e file, assegnati dalla umask di default

Fase 3 — Restrizione dei permessi con chmod

```
chmod 700 backup.sh
```

A questo punto ho applicato chmod 700, che assegna lettura, scrittura ed esecuzione solo al proprietario del file (kali) e toglie completamente ogni accesso a gruppo e altri utenti. È il modo più diretto per isolare uno script che non deve essere toccato da nessun altro.

```
(kali@kali)~[/esercizio]
$ chmod 700 backup.sh

(kali@kali)~[/esercizio]
$ ls -l backup.sh
-rwx----- 1 kali kali 26 Aug 25 08:46 backup.sh
```

Fig. 3 — Applicazione di chmod 700 e verifica dei nuovi permessi

Fase 4 — Verifica pratica delle restrizioni

```
./backup.sh
chmod u-w backup.sh
echo 'fine backup' >> backup.sh
chmod u+w backup.sh
```

Per essere sicuro che tutto funzionasse davvero, e non solo sulla carta, ho prima lanciato lo script (che stampa correttamente "Backup in corso..."), poi ho tolto anche al proprietario il permesso di scrittura con chmod u-w. Provando ad aggiungere una riga al file con echo 'fine backup' >> backup.sh, il sistema ha rifiutato l'operazione restituendo l'errore "Permission denied": segno che le restrizioni impostate vengono realmente fatte rispettare dal kernel, e non sono solo un'impostazione superficiale. Alla fine ho ripristinato il permesso di scrittura con chmod u+w.

```
(kali@kali)~[/esercizio]
$ ./backup.sh
Backup in corso...

(kali@kali)~[/esercizio]
$ chmod u-w backup.sh

(kali@kali)~[/esercizio]
$ echo 'fine backup' >> backup.sh
zsh: permission denied: backup.sh

(kali@kali)~[/esercizio]
$ chmod u+w backup.sh
```

Fig. 4 — Esecuzione dello script, revoca del permesso di scrittura e blocco della modifica

3. Riepilogo dei permessi e motivazioni

La tabella qui sotto riassume la configurazione finale e il perché di ogni scelta.

Risorsa	Permessi	Significato
Cartella esercizio	775 drwxrwxr-x	Proprietario e gruppo hanno pieno accesso (rwx): possono creare, cancellare e attraversare la cartella. Gli altri utenti hanno solo r-x, quindi possono guardare cosa c'è dentro ma non modificare nulla.
File backup.sh	700 -rwx-----	Solo il proprietario (kali) può leggere, modificare ed eseguire lo script. Gruppo e altri non hanno alcun permesso: niente lettura, niente scrittura, niente esecuzione.

In sintesi: trattandosi di uno script che potrebbe contenere informazioni o comandi delicati, ha senso che solo il proprietario possa leggerlo, modificarlo o eseguirlo. In questo modo si evita sia che qualcun altro lo legga (information disclosure) sia che lo esegua senza autorizzazione.

4. Conclusioni

I test svolti confermano che Linux applica correttamente le policy dei permessi POSIX: l'errore di Permission denied ottenuto nella Fase 4 dimostra che le restrizioni impostate con chmod non sono solo teoriche, ma vengono effettivamente fatte rispettare dal sistema. L'esercizio mi ha aiutato a capire meglio come il principio del privilegio minimo si traduca in pratica in comandi semplici come chmod, e quanto sia importante controllare sempre i permessi di file sensibili come gli script di sistema.